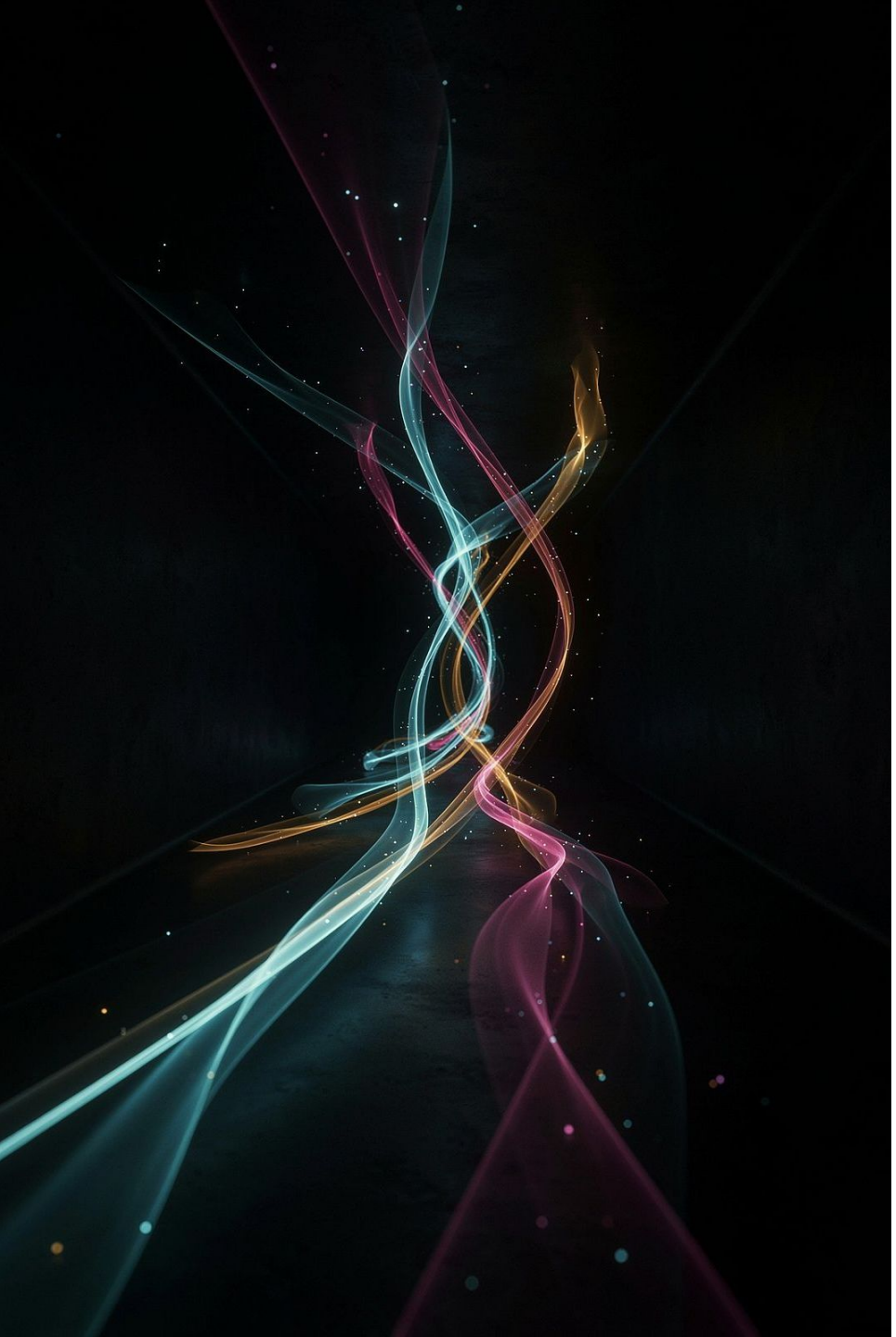


Series temporales y redes recurrentes

Sprint 12 · Series, lenguaje y generativa · S12.1

La dirección de TurisData quiere prever la ocupación de los próximos 7 días. El tiempo importa: el modelo no puede ver el futuro.

Mirar la serie antes de modelar

Tres cosas que buscamos en ocupacion_diaria.csv:

Tendencia

¿Sube o baja a largo plazo?

Estacionalidad

Patrones que se repiten: la semana, el año.

Ruido

Lo que no sigue ningún patrón.

```
dia_max = por_dia.idxmax(); mes_min = por_mes.idxmin()      # Ejercicio 1
```

Partición temporal: ¡sin barajar!

- Con tablas partimos al azar. Con series no: entrenamos con el pasado y evaluamos con lo último.
- Cuaderno S12_01: los últimos 91 días (13 semanas) son el test; previsión a 7 días repetida cada semana.
- Barajar antes de partir sería fuga: el modelo vería el futuro. Es la misma idea de «sin fuga» de siempre.

Baselines: el rival que hay que batir

Baseline	Idea
Ingenuo	mañana igual que hoy (último valor conocido)
Estacional ingenuo	el lunes que viene como el último lunes
Media de 4 semanas	media de los últimos 4 días iguales (Ejercicio 2: pred_media4)

Un modelo complejo solo vale la pena si gana a algo muy simple. A veces el baseline es la respuesta correcta: más simple y más fácil de explicar.

Descomposición y SARIMAX

serie = tendencia + estacionalidad + resto (seasonal_decompose, ciclo de 7 días).

```
amplitud_semanal = desc.seasonal.max() - desc.seasonal.min()     # Ejercicio 3
mae_sin_exog = ...SARIMAX(entrenamiento, order=(1, 0, 1),
                          seasonal_order=(1, 1, 1, 7))          # Ejercicio 4
```

AR: depende de días anteriores. I: diferencias para quitar tendencia. MA: depende de errores recientes. S: lo mismo cada 7 días. Se añaden festivos y Fourier como variables exógenas.

Mini-reto: mejora_pct = 100 * (mejor_baseline - mae_sar) / mejor_baseline.

Redes recurrentes: una red con memoria (S12_02)

Una red normal mira cada fila por separado. Una recurrente lee la secuencia paso a paso y actualiza un estado (resumen de lo visto).

Capa	Idea
SimpleRNN	resumen simple; olvida lo lejano
LSTM	puertas que deciden qué recordar y qué olvidar
GRU	versión más ligera de LSTM

Entrada de cada día: ocupación, festivo y día de la semana/año como seno/coseno (así domingo y lunes quedan cerca).

Ventanas deslizantes y escalado sin fuga

```
def crear_ventanas(origenes):                    # Ejercicio 2
    X = [matriz[t - VENTANA:t] for t in origenes]     # (n, 28, 6)
    y = [objetivo[t:t + HORIZONTE] for t in origenes]  # (n, 7)
    return np.array(X), np.array(y)
```

- Entrada = días t−28 … t−1; salida = días t … t+6.
- El escalador se ajusta (fit) solo con el entrenamiento; con la prueba solo transform.

La pregunta importante

No es «¿cuál red gana?» sino «¿gana alguna al baseline?»

- Ejercicio 3: entrenar "SimpleRNN", "LSTM" y "GRU" (32 unidades, early stopping) y comparar su MAE en puntos de ocupación.
- Con ~1000 días de datos y un patrón claro, un baseline estacional o SARIMAX pueden ganar a la red.
- Mini-reto: GRU con ventana de 7 días (mae_ventana7): ¿cuánto empeora?

Ahora, a practicar

Lo que viene a continuación está en tu repositorio.

CUADERNO · Series temporales: baselines, descomposición y SARIMAX
(sprints/S12/notebooks/S12_01_series_temporales) — 45 min.

CUADERNO · RNN, LSTM y GRU frente al baseline
(sprints/S12/notebooks/S12_02_redes_recurrentes) — 45 min.